

Notes Week 26

a. Communication with Zara

Dear Zara,

My name is Koen Essers and I am a master's student working with Onno Pols at Radboud University. Onno recently contacted Amanda about intershell abundances of TPAGB stars, and you were kind enough to share your data with us.

Thank you very much for this! I think it will be very useful for our project. Onno mentioned that you offered to help if I ran into any questions regarding the data, so I hope you don't mind me reaching out with two questions:

At the beginning of every thermal pulse in the intershell `z..._pmz..._m...` .dat files, there is a line containing several values. For example: `15 2.4504e+09 2.4504e+09 1.48432 0.622382 99649 197.67 0.021343` I assume that here, the first seven values correspond to the TP count, the age of the star, the age again, the mass of the star, the core mass, the interpulse duration, and the pulse duration. I am, however, not sure what the last number represents. Could you confirm that my assumptions are correct and tell me what the last number is? I noticed that, for every thermal pulse, the abundances are listed multiple times. Are these the abundances at different times during a thermal pulse? If so, could you tell me whether these different timesteps are chronologically ordered, and when the abundances are calculated for the first and last time during a pulse?

Thank you again for sharing the data and for being willing to help.

Best regards, Koen Essers

Me

Hi Koen,

I am happy to help you where I can. What is your masters project on?

The seven values are: thermal pulse count, simulation time X2 (years, they might be the start/end time of the pulse, but it is too short to make any difference here), total mass (msun), core mass (msun), interpulse duration (years), pulse duration (years), and Helium shell mass (msun). If you have access to Amanda's pulse.dat files (they can probably be accessed online), you should see this information (as well as a few other parameters).

As for the multiple abundances, they are listed chronologically. As for when they are calculated, I don't believe there is a specific trigger. It is calculated from models saved by Amanda Karakas. I don't recall the frequency. Unfortunately, I no longer have access to her models. However, Amanda might be able to verify this for you.

All the best,

Zara

Zara

Hi Zara,

Thanks for your response. This clears up my questions.

The project is about modeling mass transfer from TP-AGB stars in binary systems. My advisor and I specifically want to see if we can reproduce some of the observed properties of barium stars. We use MESA for the binary simulations using only a very limited number of nuclear reactions. My advisor and I hope that we can use your detailed intershell abundances together with the dredge-up information from MESA to estimate the envelope abundances of our MESA TPAGB models. With this, we then hope to estimate the enrichment of the companion star after mass transfer.

To match the intershell abundances of a given thermal pulse in the Monash models with the corresponding pulse in a MESA model, it would be really helpful to have the Δ MDUP information for each thermal pulse. I suspect this information might be in the pulse.dat files as well.

Unfortunately, I could not find the pulse.dat files easily online, so I will ask Amanda about them.

Thanks again for your help!

Best regards, Koen

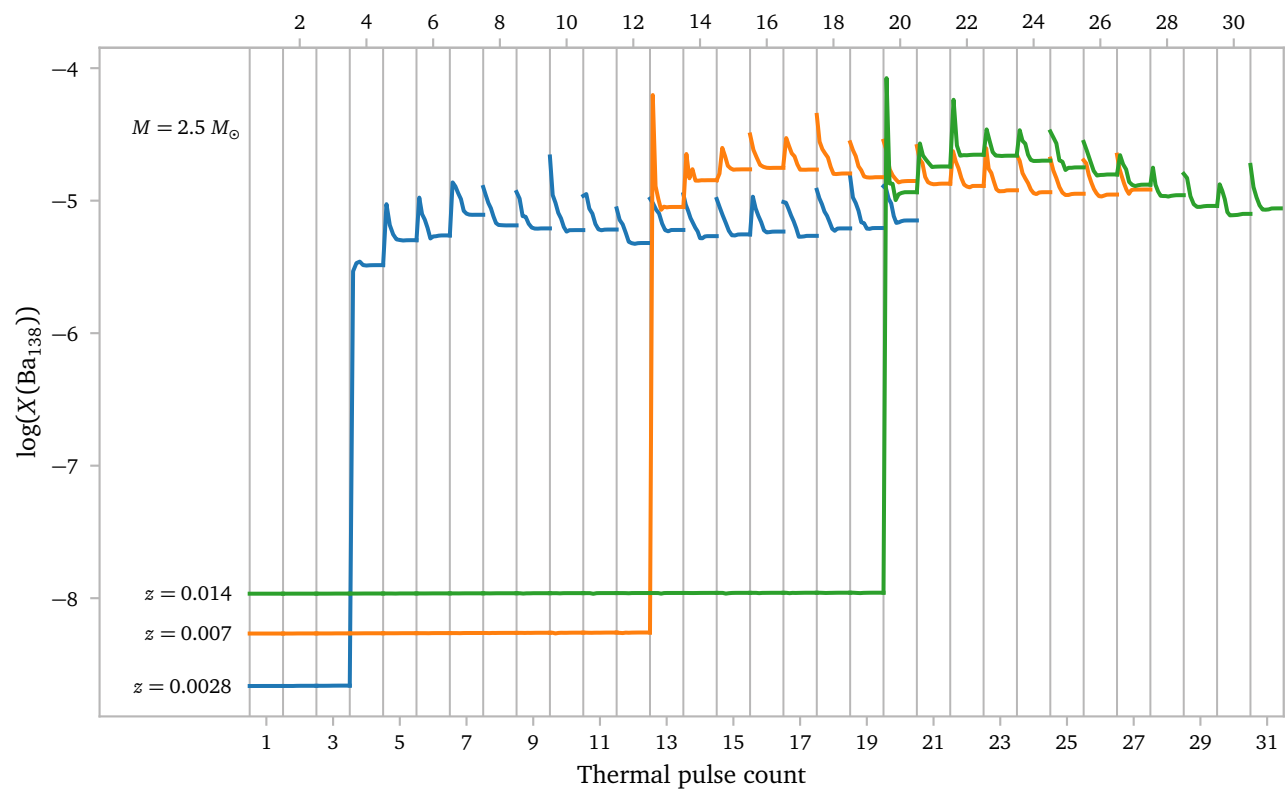
Me

Zara confirmed that the different sets of isotopes per pulse are indeed the abundances at different times. She could not confirm *when* the abundances are calculated.

At this point I felt comfortable to look some more at the intershell values.

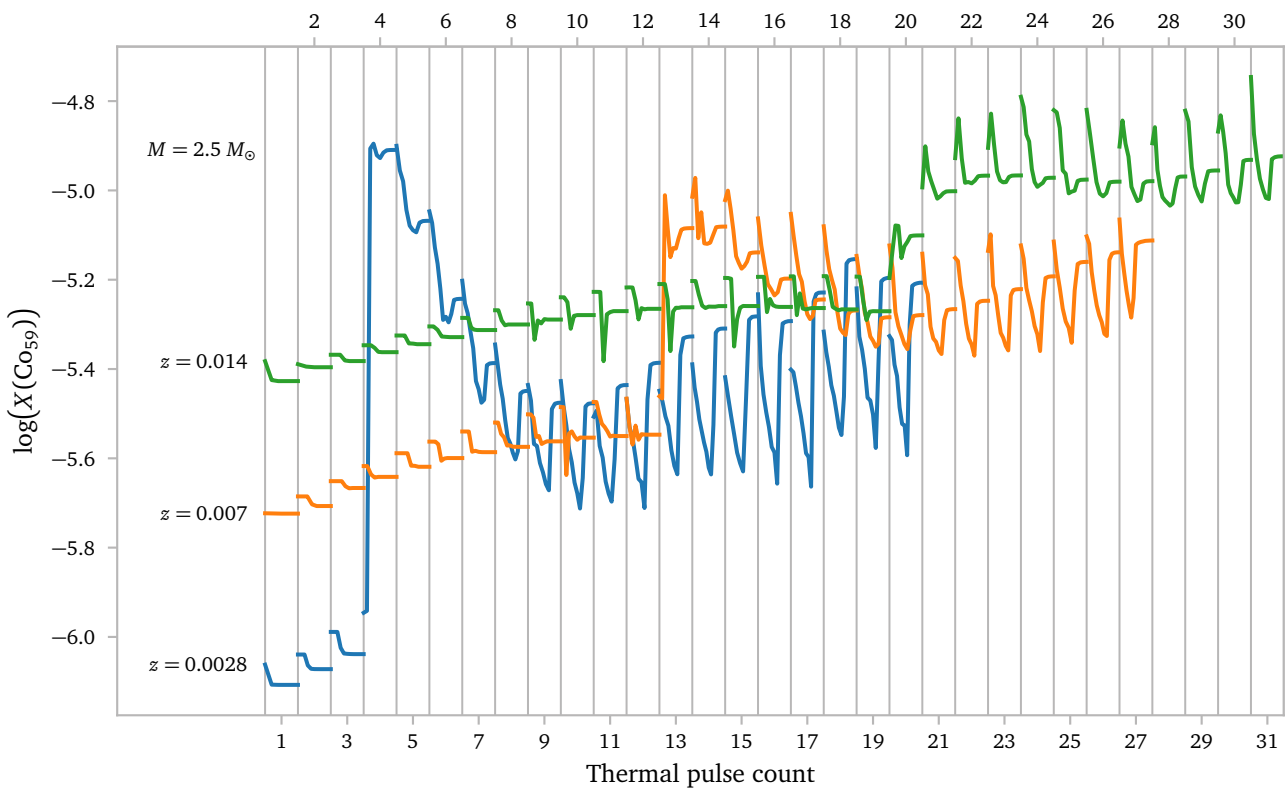
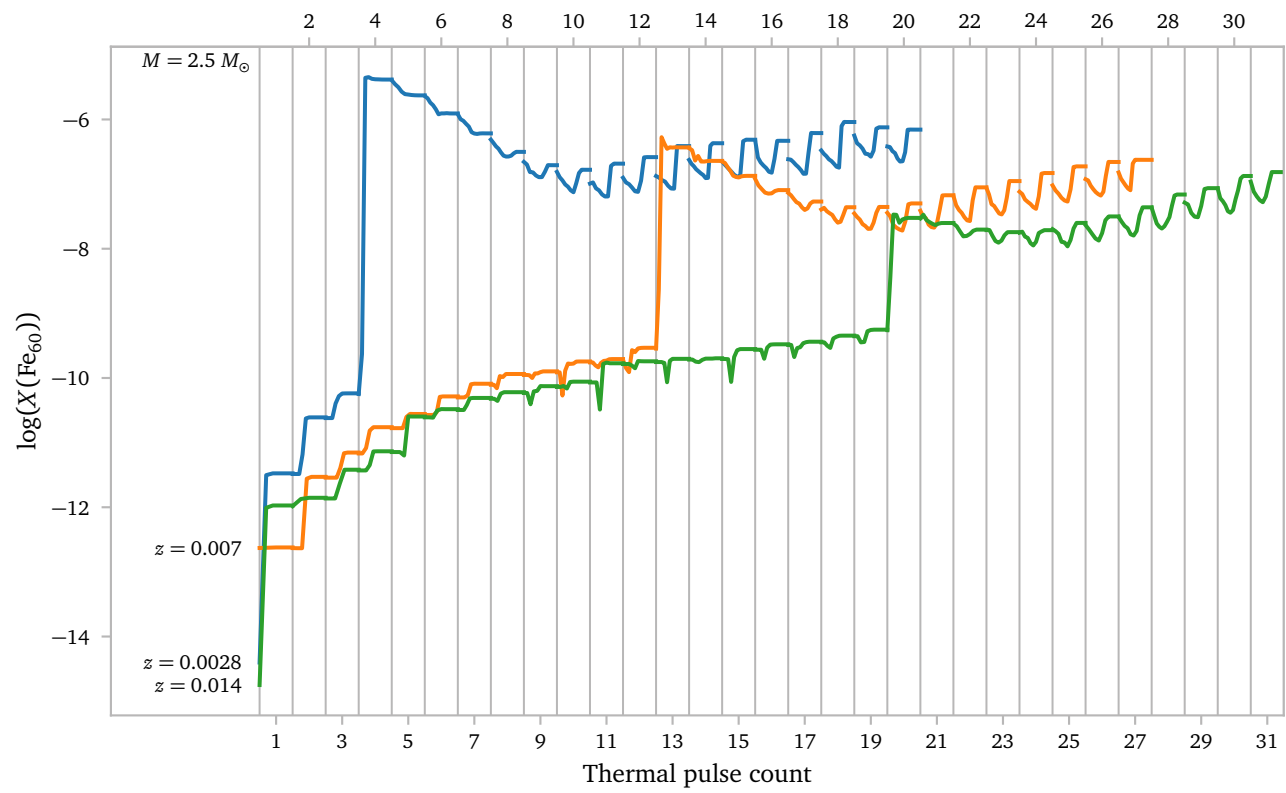
b. Intershell

The plot below shows the effects of metallicity on the Ba_{138} abundances in the intershell for a $1.5 M_{\odot}$ TPAGB star.

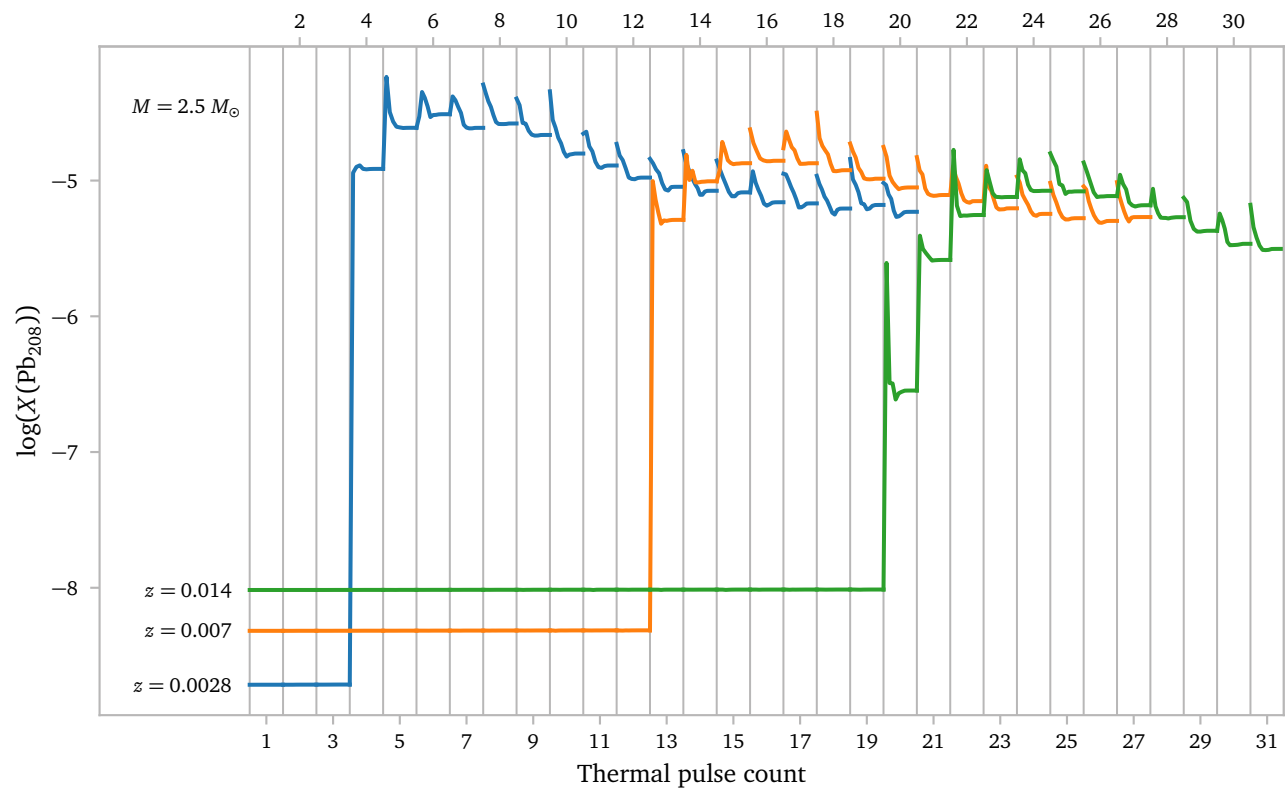


It looks like the metallicity changes the first time dredge-up occurs. This is expected as well, since we see that lower metallicity stars have more dredge-up than the higher metallicity ones (Karakas & Lugaro, 2016).

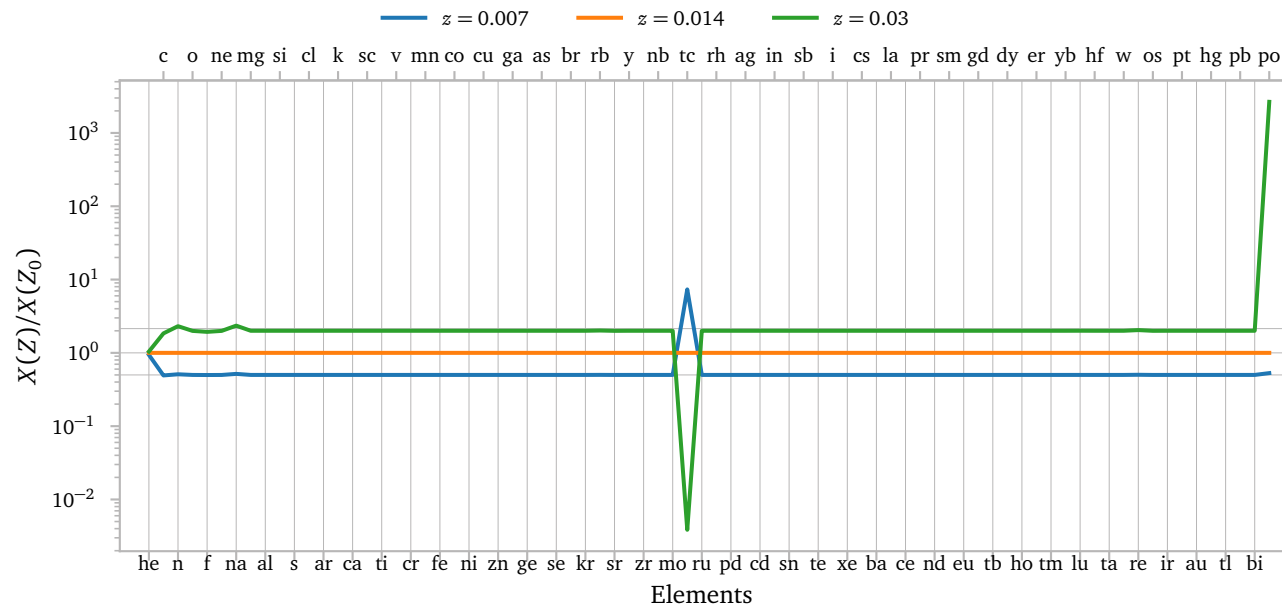
Now let me look at two elements that are very quickly produced by the s-process.



Now I plot Pb_{206} , the heaviest stable isotope that is produced by the s process

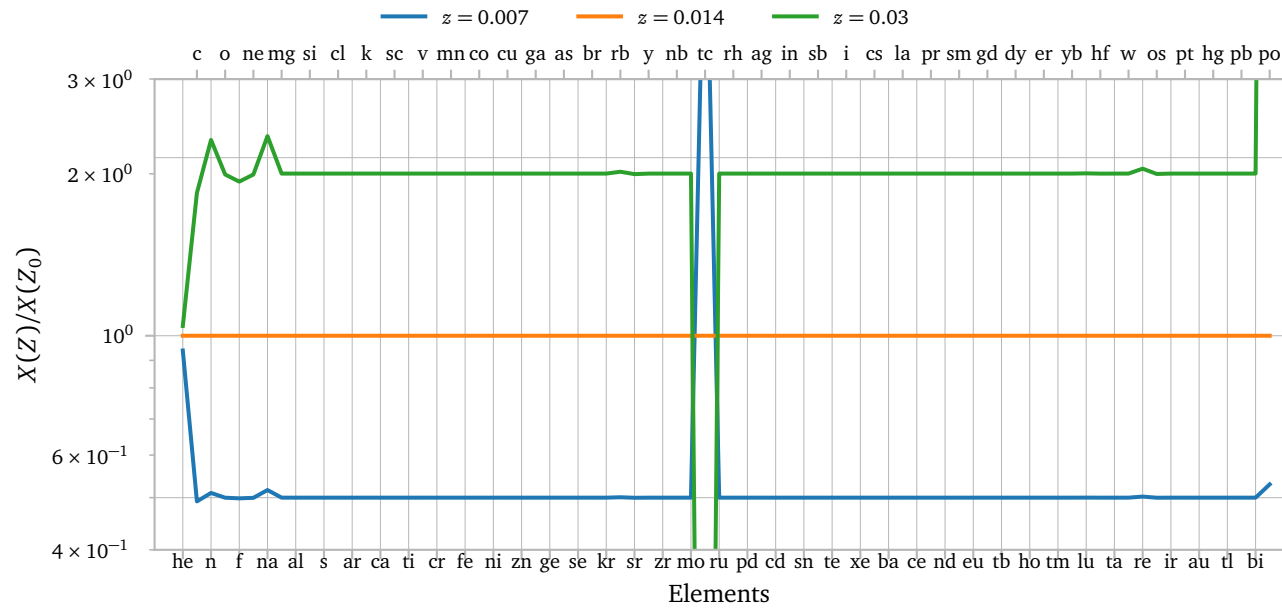


Here I plot the metallicity dependence on the initial envelope abundances of a $1.5 M_{\odot}$ star. This means there is no TDU enrichment. The vertical axis shows the abundances scaled to the solar metallicity model.



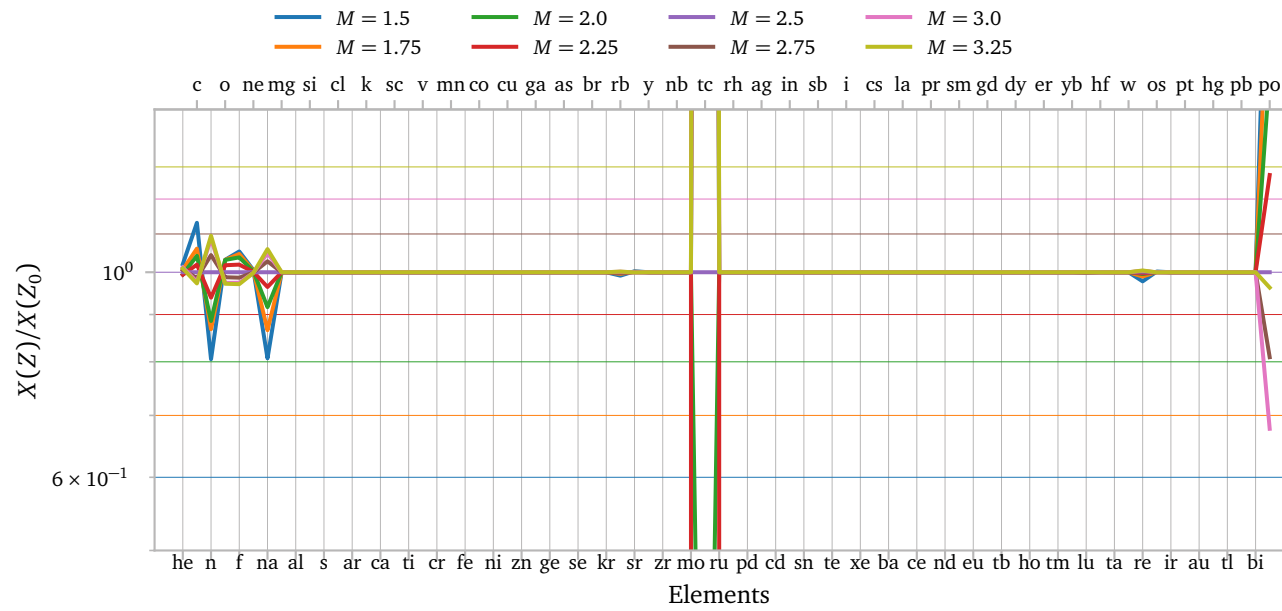
The horizontal lines in this figure show $Z/Z_{\odot}$. Clearly, nearly all elements are just linearly scaled with metallicity. We do see however that the Helium abundances, technetium abundances and the polonium abundances differ. I think this is not a big problem. It just means we should be careful when we analyse the enrichment of these elements.

Below, I show the same figure, now with a zoomed in vertical axis.



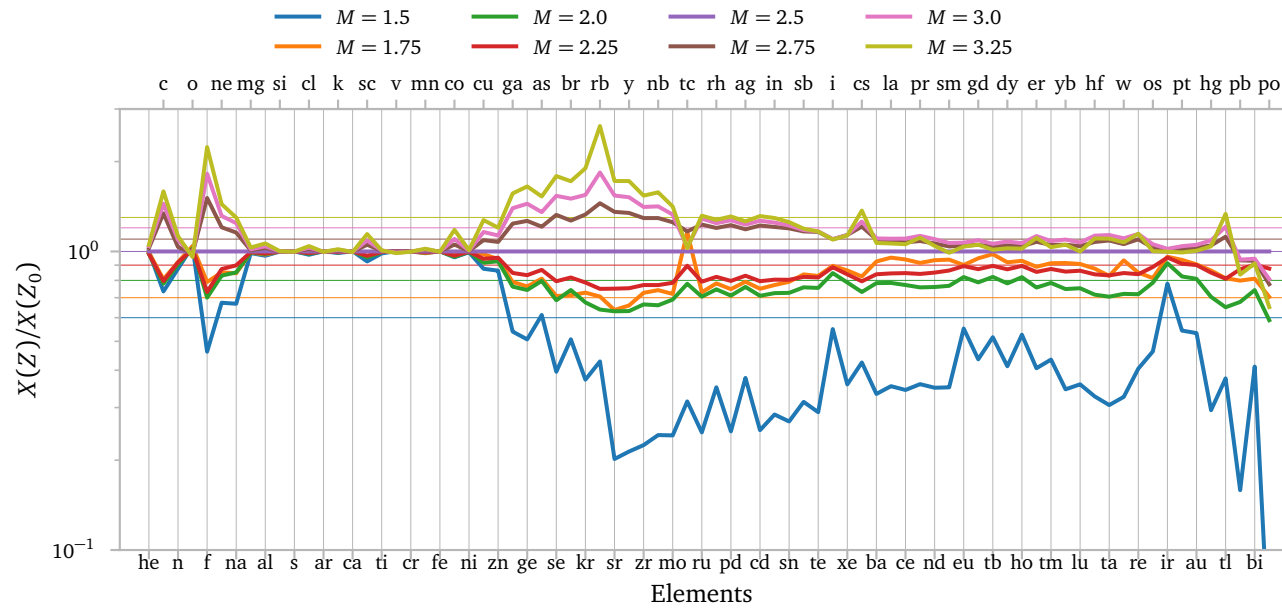
I think we can clearly see here that the $Z = 0.03$ model is scaled by a slightly different value. In fact, it looks like the abundances are nearly all at exactly 2, which would make sense and mean that $Z = 0.028$ or $2Z_{\odot}$.

We can do the same, but now looking at the mass:



We see that for the light elements, there is some variation with mass, *but*, nearly all *s*-process elements have initial abundances that are *independent* of the initial mass of the star.

If we oppose this to the final abundances, we see a clear mass dependence, although not linear and not equal per element.



Anyway, we can simply use the metallicity and linearly scale this with a standard envelope abundance to get the envelope abundances for our *MESA* models.

c. Thermal Pulse information

I asked Amanda for this information and I got a bunch of separate files.

Dear Amanda,

My name is Koen Essers and I am a master's student working with Onno Pols at Radboud University. Onno recently contacted you about intershell abundances of TP-AGB stars, and you kindly shared this data with us.

Thank you very much for this. I think it will be very useful for my project.

The project is about modelling mass transfer from TP-AGB stars in binary systems with the aim of reproducing some of the observed properties of barium stars. We use MESA for the binary simulations and would like to combine the dredge-up information from MESA with the detailed intershell abundances from the Monash models to estimate the enrichment of a companion star after mass transfer.

I have a few questions about the data that I was hoping you could help me with.

1. Zara mentioned that, besides the intershell abundances, surface abundances and yields data, there are also pulse.dat files that contain information about the individual thermal pulses of the Monash models. I was not able to find these files online. Could you point me towards where these pulse.dat files for the $z=0.0028$, $z=0.007$ and $z=0.014$ Monash models can be accessed?

In particular, having the ΔMDUP information for each thermal pulse would be very useful for my project, as I would like to match the intershell enrichment of a given thermal pulse in the Monash models with the corresponding pulse in a MESA model.

2. I noticed that, for every thermal pulse, the abundances in the intershell_z..._pmz..._m....dat files are listed multiple times. Zara mentioned that these are chronologically ordered and based on saved models. Could you tell me whether the last set of abundances saved for a pulse is before or after the third dredge-up event that follows a pulse?

Thanks in advance for your help,

Best regards, Koen Essers

Me

Hi Koen,

Thank you very much for this. I think it will be very useful for my project.

No problems, happy to help out.

The project is about modelling mass transfer from TP-AGB stars in binary systems with the aim of reproducing some of the observed properties of barium stars. We use MESA for the binary simulations and would like to combine the dredge-up information from MESA with the detailed intershell abundances from the Monash models to estimate the enrichment of a companion star after mass transfer.

I have a few questions about the data that I was hoping you could help me with.

Zara mentioned that, besides the intershell abundances, surface abundances and yields data, there are also pulse.dat files that contain information about the individual thermal pulses of the Monash models. I was not able to find these files online. Could you point me towards where these pulse.dat files for the $z=0.0028$, $z=0.007$ and $z=0.014$ Monash models can be accessed?

I can provide these easily. I will share a file with you that should contain all of the pulse files (which contain pulse-by-pulse information for all of the AGB models). If it doesn't contain this information please let me know (I haven't checked the .zip file for a while but I think it contains the files you want).

In particular, having the DeltaMDUP information for each thermal pulse would be very useful for my project, as I would like to match the intershell enrichment of a given thermal pulse in the Monash models with the corresponding pulse in a MESA model.

this information should be in the pulse.dat files!

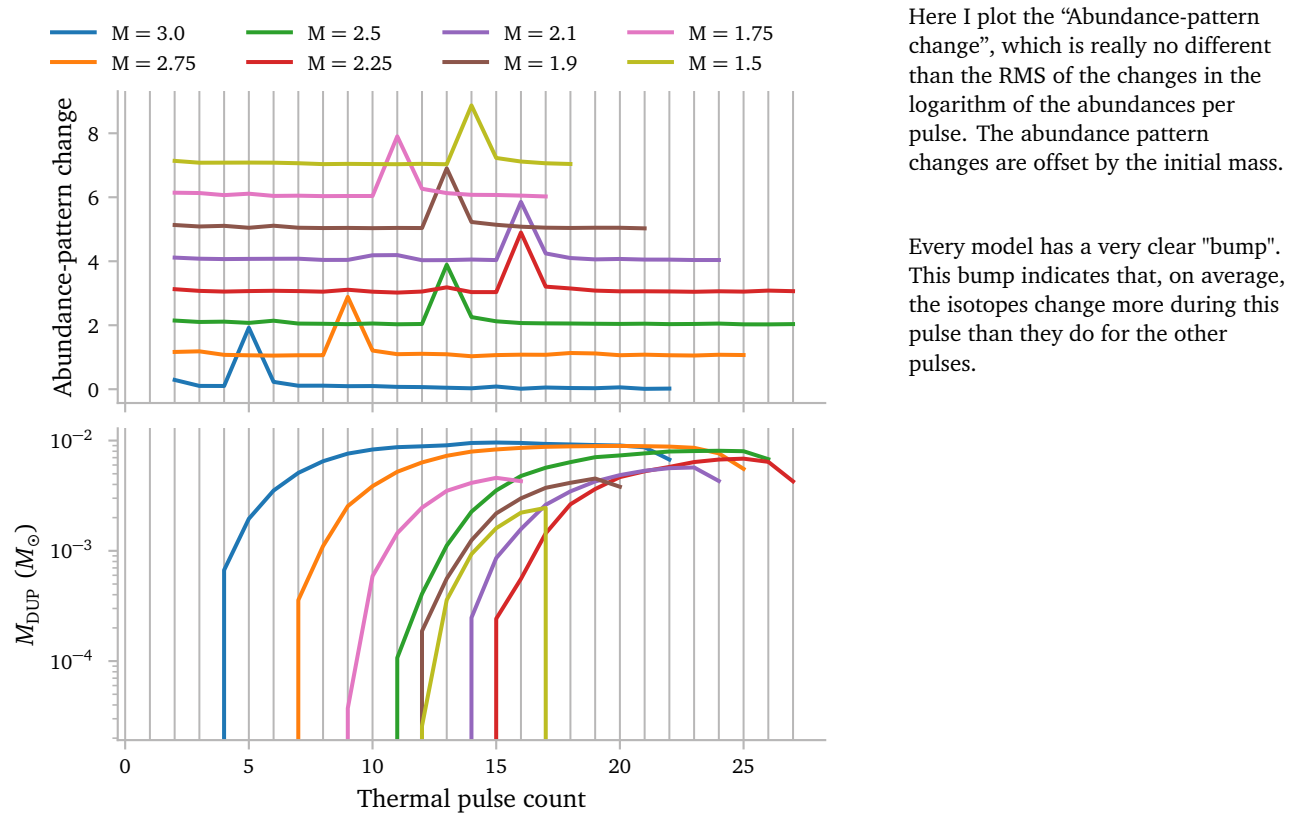
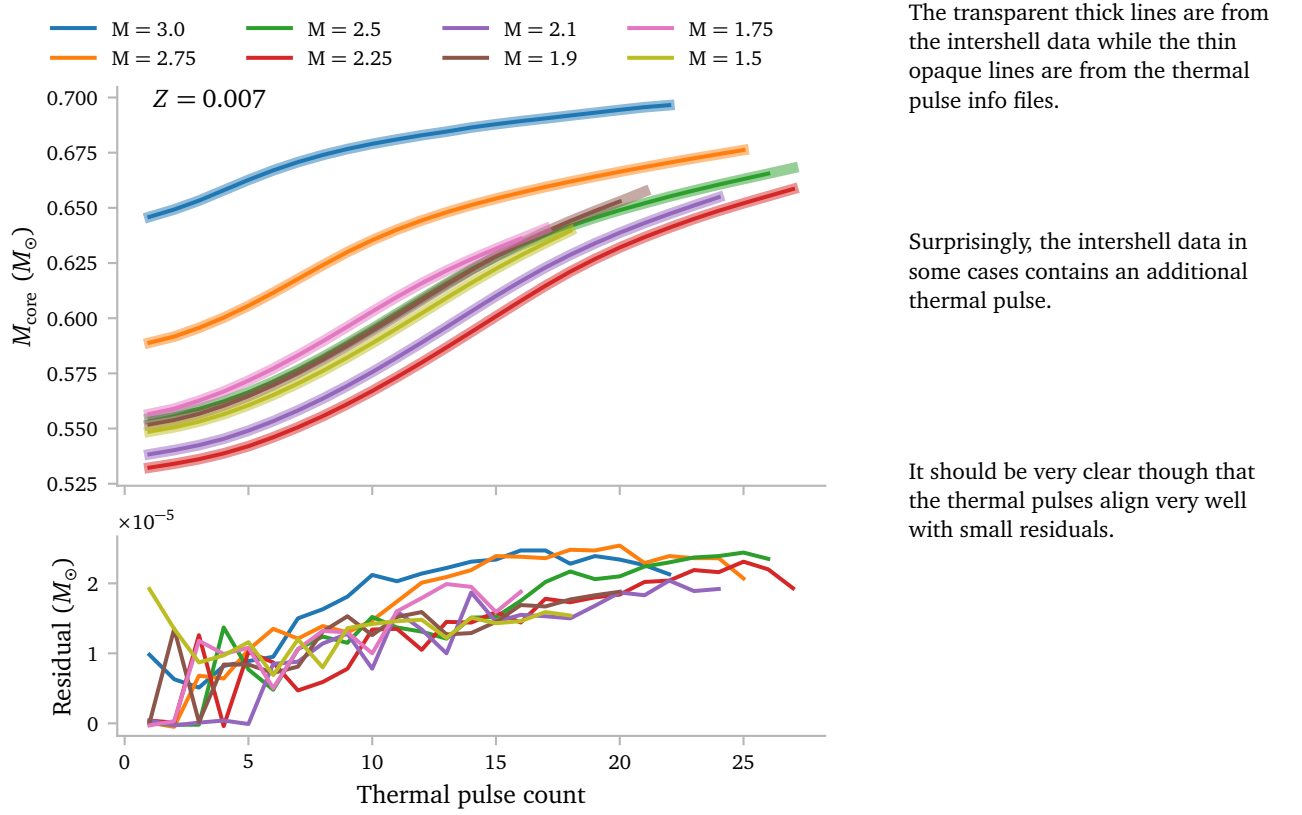
I noticed that, for every thermal pulse, the abundances in the intershell_z..._pmz..._m....dat files are listed multiple times. Zara mentioned that these are chronologically ordered and based on saved models. Could you tell me whether the last set of abundances saved for a pulse is before or after the third dredge-up event that follows a pulse?

definitely before a thermal pulse. the data files are sourced from the intershell, when intershell convection is active. It is a good time to "sample" the composition because convection homogenised the intershell area (except for perhaps part of the H-shell). Intershell convection dies down before a third dredge-up begins.

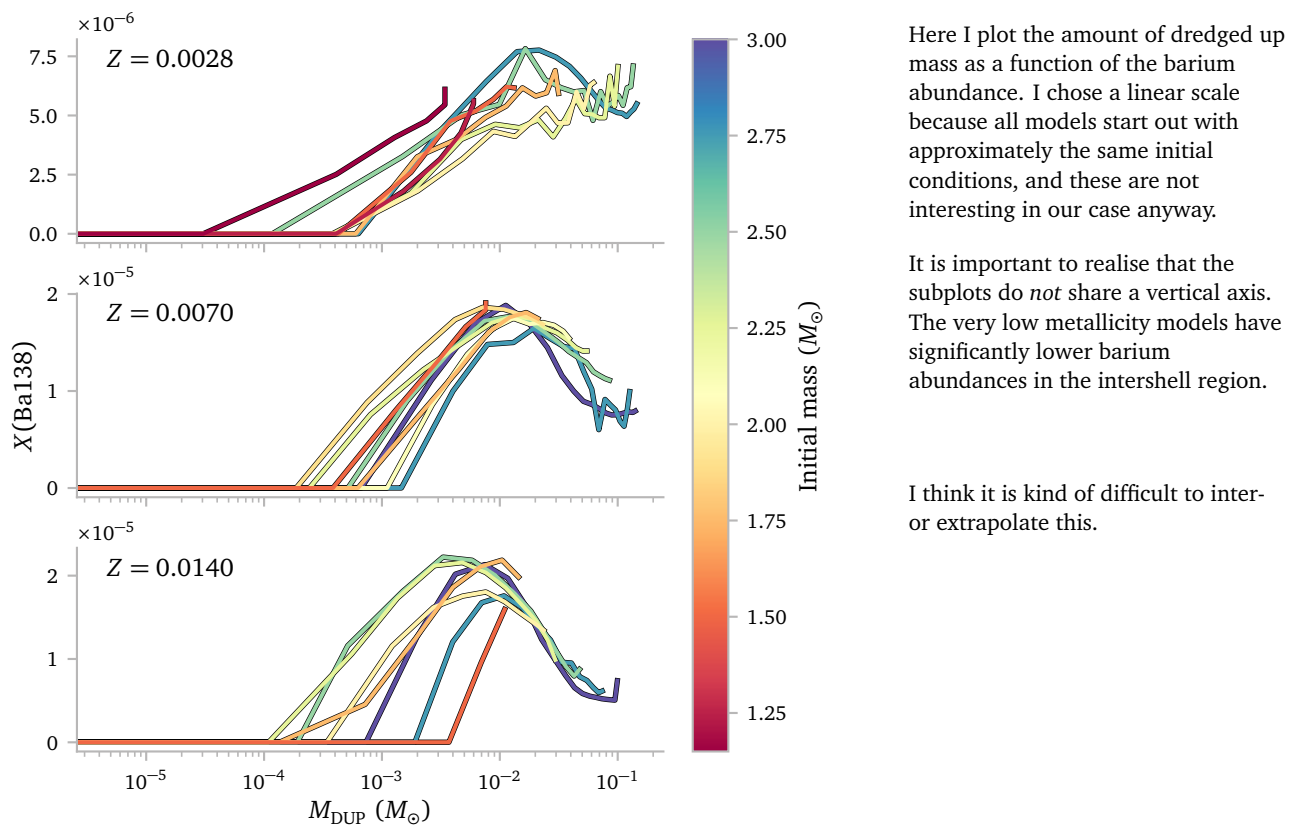
I hope that helps! Amanda

Amanda Karakas

I want to confirm that the thermal pulses of the thermal pulse information files and the intershell abundance files correspond. The easiest way to do this is by looking at a quantity that is in both datasets. In this case I chose the core mass:



In the Appendix A I show the same figure but now for other metallicities.



d. Interpolating Envelope data

I use trilinear interpolation. Firstly, I interpolate the elemental abundance and the dredged up mass for a specific Monash model. I do this interpolation in log space, which I think makes most sense here.

```
def get_iso_and_Mdredge(isotope, intershell, tp_info):
    # keep only pulses that exist in both datasets
    intershell = intershell[intershell.ntp.isin(tp_info.pulse)]

    # one intershell abundance per pulse
    i_data = intershell.drop_duplicates("ntp").set_index("ntp")

    # one dredge-up mass per pulse
    tp_data = tp_info.drop_duplicates("pulse").set_index("pulse")

    # common pulses, in ascending order
    pulses = i_data.index.intersection(tp_data.index).sort_values()

    Mdredge = tp_data.loc[pulses, "Ddredge"].to_numpy()
    isotope_abundance = i_data.loc[pulses, isotope.key].to_numpy()

    Mdredge = np.log10(np.cumsum(Mdredge) + 1e-12)
    isotope_abundance = np.log10(isotope.mass * isotope_abundance)

    return isotope_abundance, Mdredge
```

and then the interpolation:

```
iso, Mdredge_real = get_iso_and_Mdredge(isotope, intershell_data, thermal_pulse_data)
abundance = np.interp(np.log10(Mdredge_interp), Mdredge_real, iso)
```

I then linearly interpolate these abundances between two masses that are closest in the dataset. This is done in linear space.

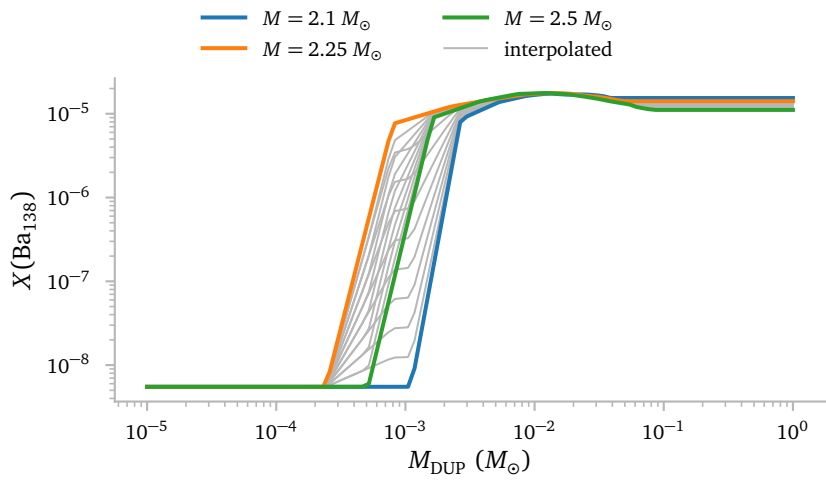
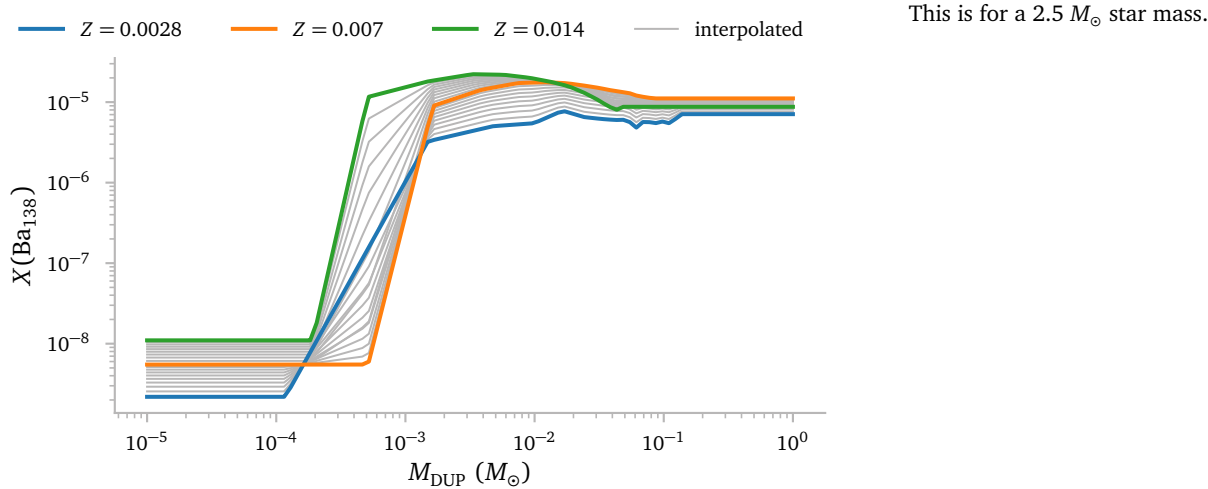
```
weight = (M - ms[arg_min]) / (ms[arg_max] - ms[arg_min])
return abundance_min + weight * (abundance_max - abundance_min)
```

Lastly, I interpolate the interpolated masses for different metallicities. I do this in log space again.

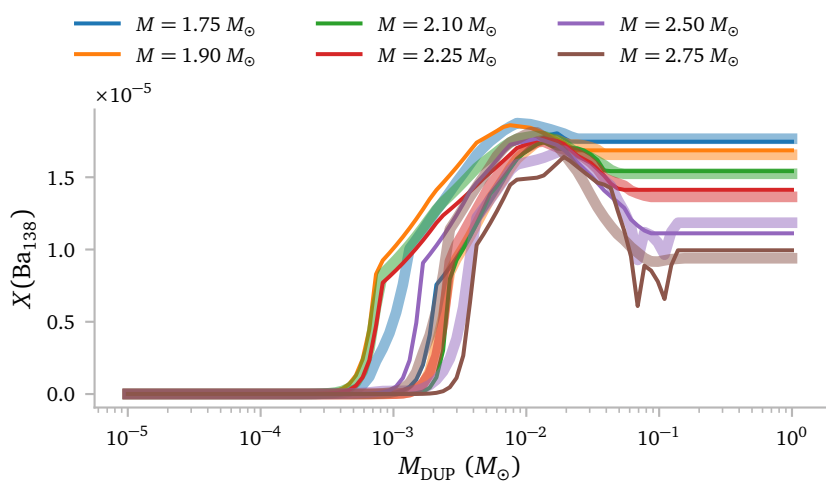
```
abundance_min = get_abundance_Z(isotope, M, z_min, Mdredge_interp)
abundance_max = get_abundance_Z(isotope, M, z_max, Mdredge_interp)
weight = (np.log10(Z) - np.log10(z_min)) / (np.log10(z_max) - np.log10(z_min))

return 10 ** (abundance_min + weight * (abundance_max - abundance_min))
```

To show it working, here I plot some interpolated values for the barium abundance.



I also checked the interpolation results with the actual values.



Here, the opaque thin lines are the actual values. The thick transparent lines are the interpolated values if we remove the actual value.

It looks like the interpolator has some trouble predicting exactly when the first barium enrichment occurs. I do not think this is super important though, since the envelope will become enriched over many pulses. What is more important is that the final values are somewhat accurate, which they seem to be.

e. Abundances Code

Now I am armed with everything I need to do some abundance calculations with my *MESA* models.

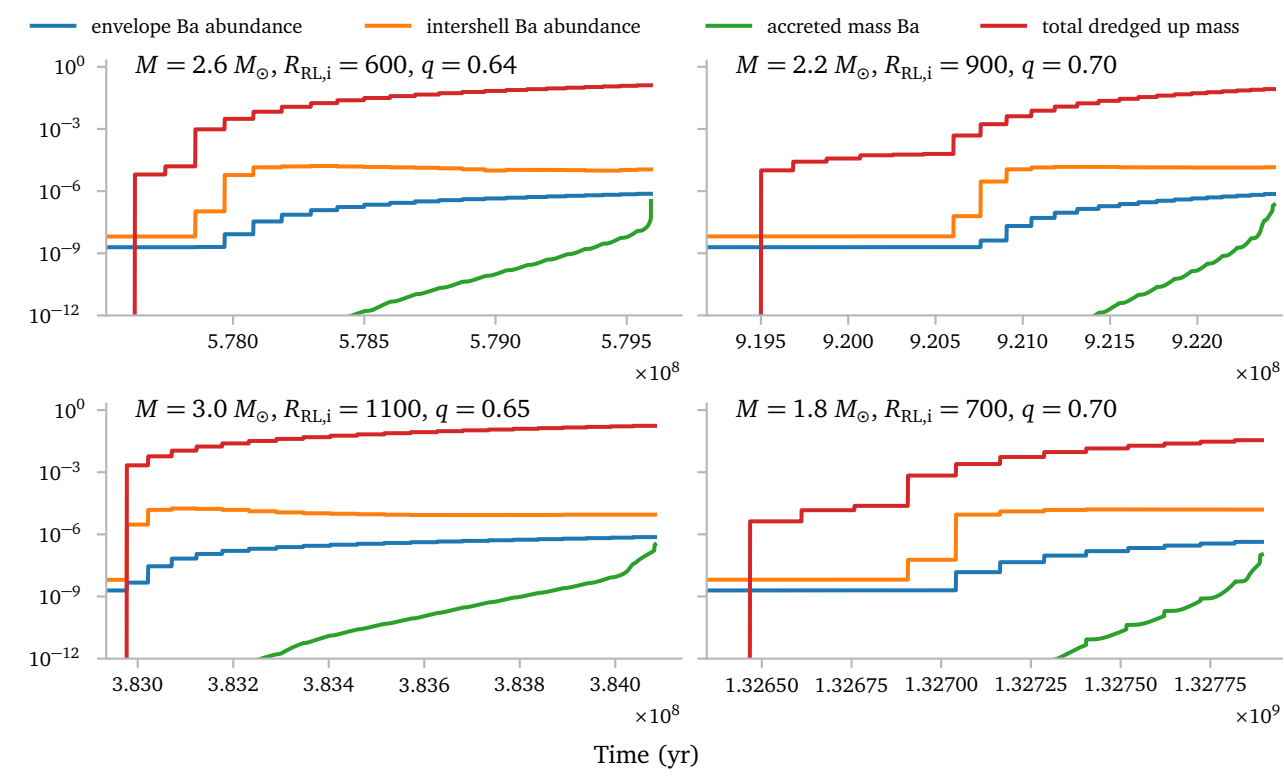
I have:

- Initial envelope abundances
- Intershell abundances as a function of dredged up mass, initial mass and metallicity
- Detailed *MESA* binary models with dredged up mass, and mass loss
- Simple binary models and single star *MESA* models.

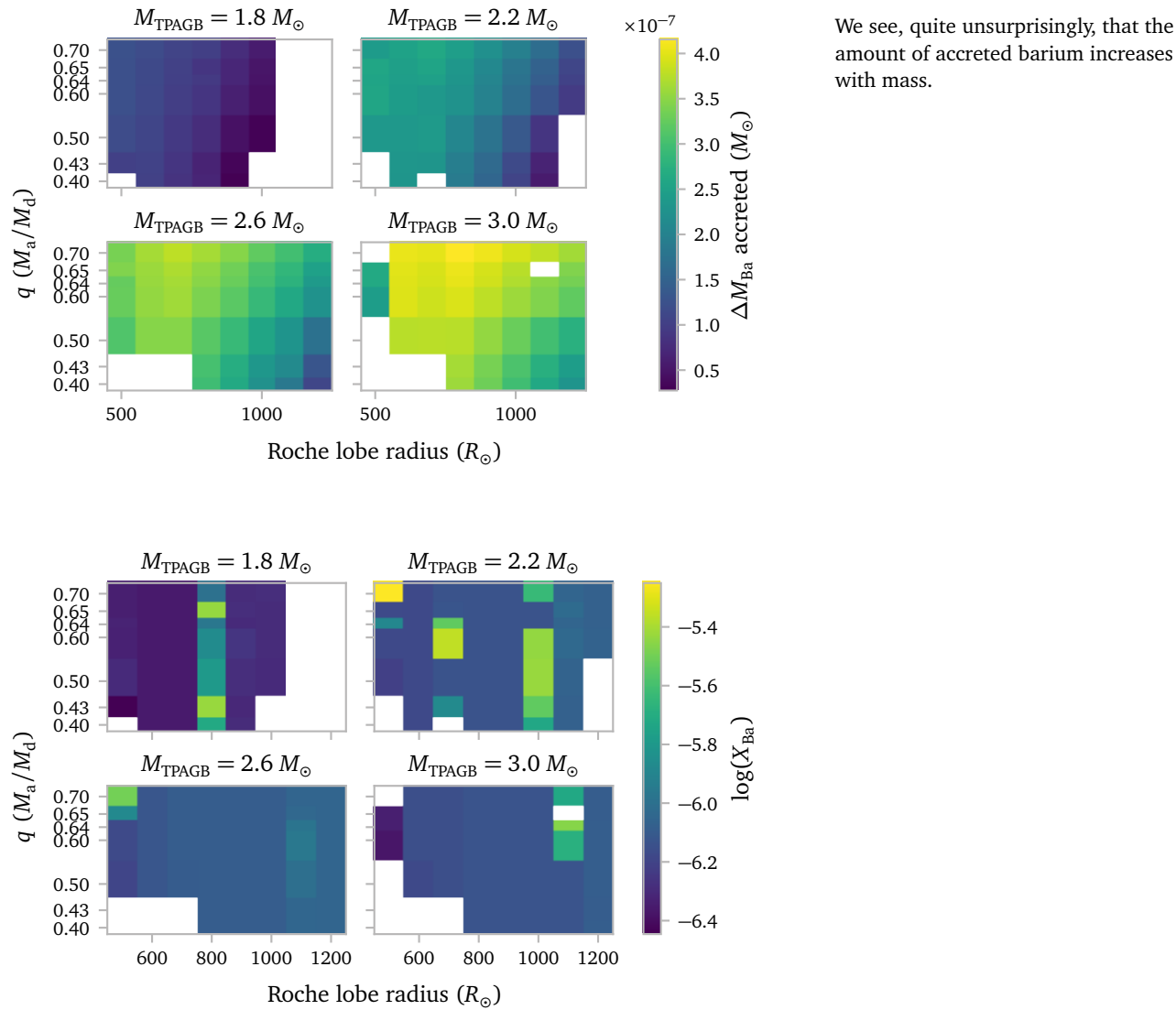
The trick is to combine this data in the correct way.

My implementation can be seen in the appendix.

Here is an example of for 4 *MESA* binary models that show what my code is capable of.



We can also compute, for example, how much barium is accreted onto the companion for every star:

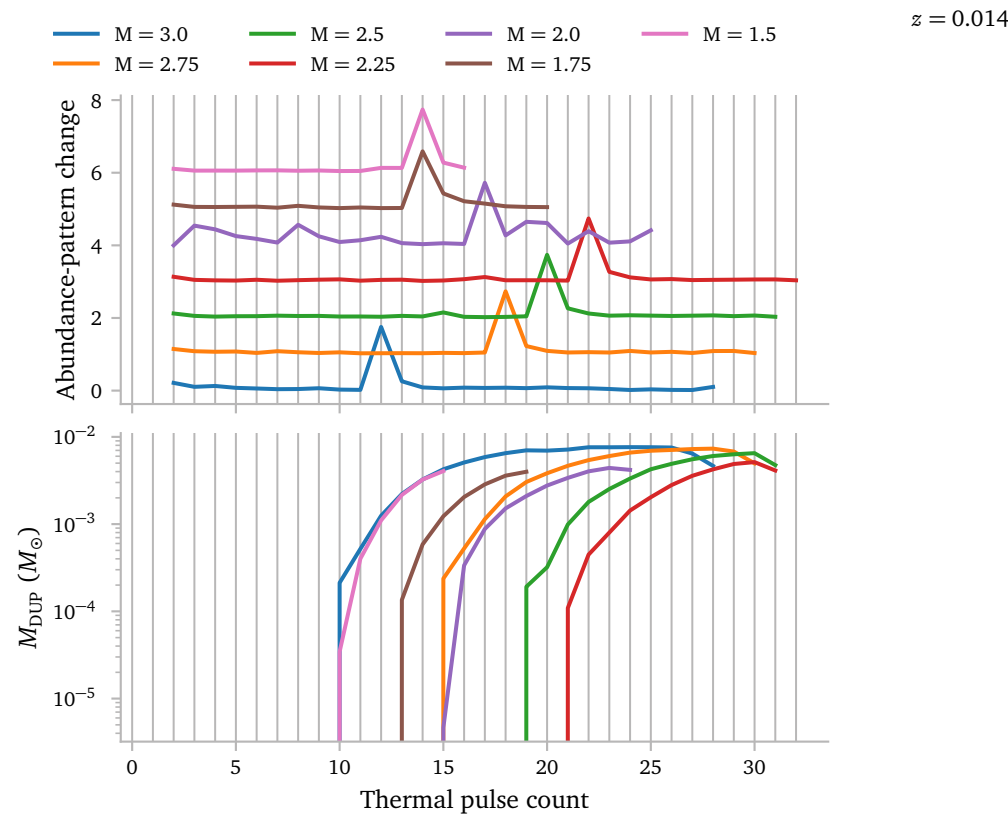
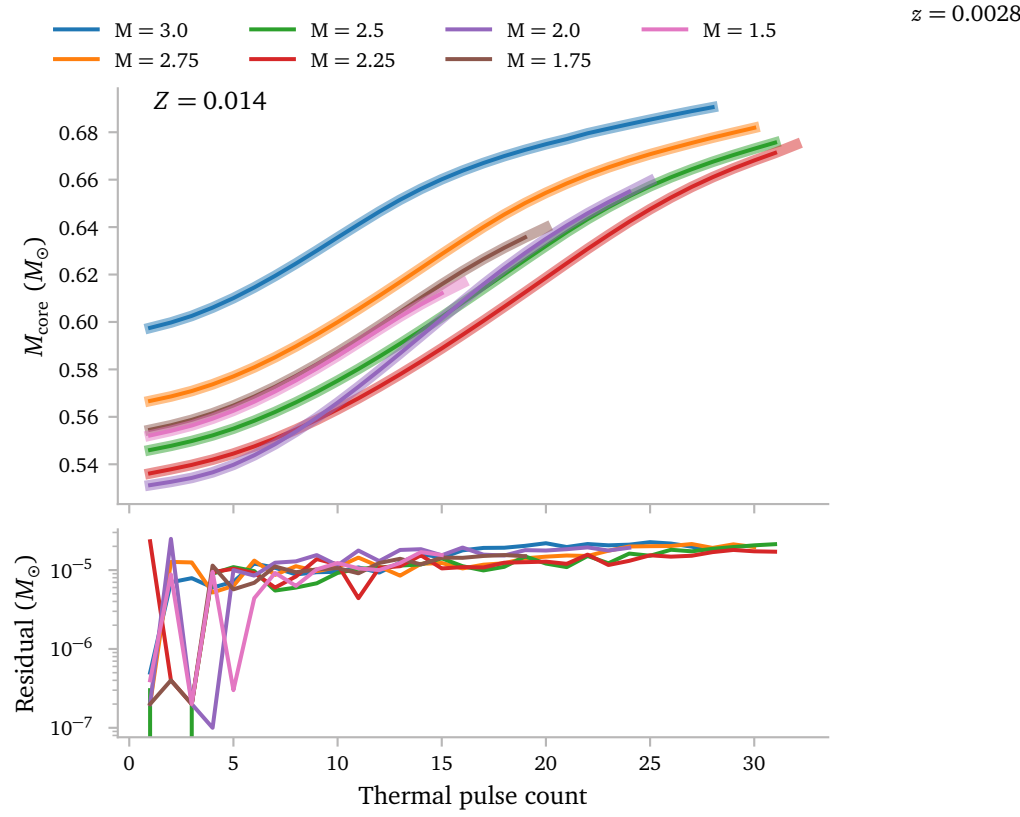
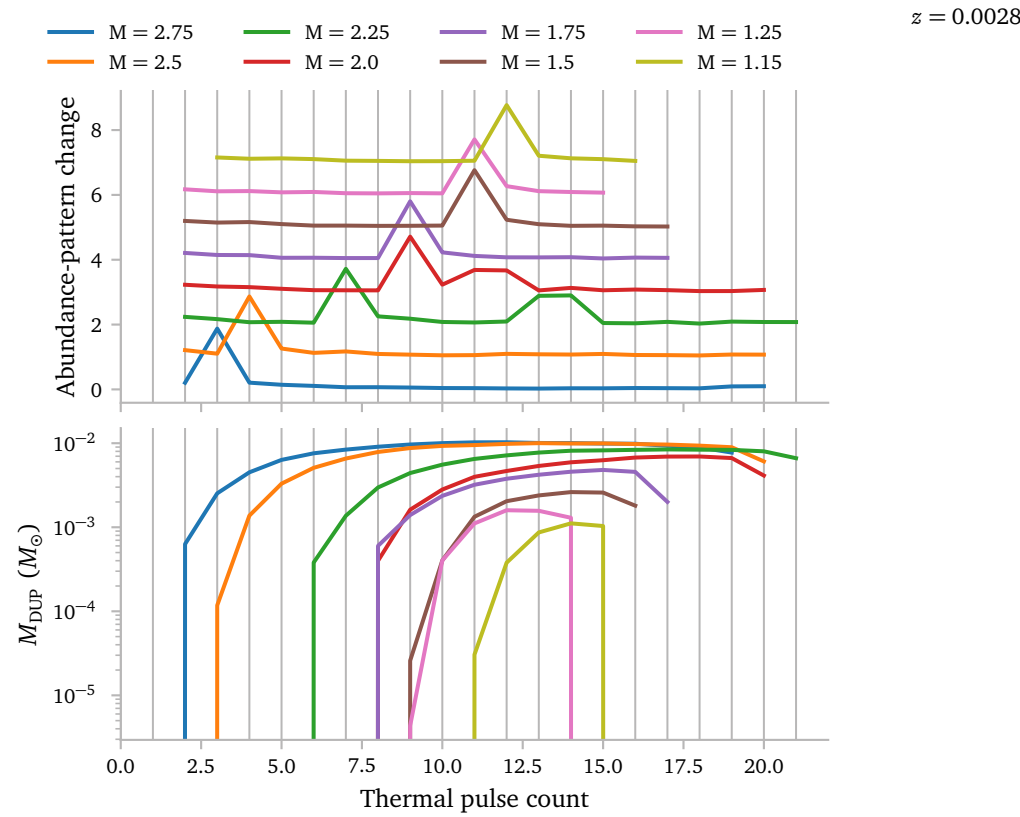
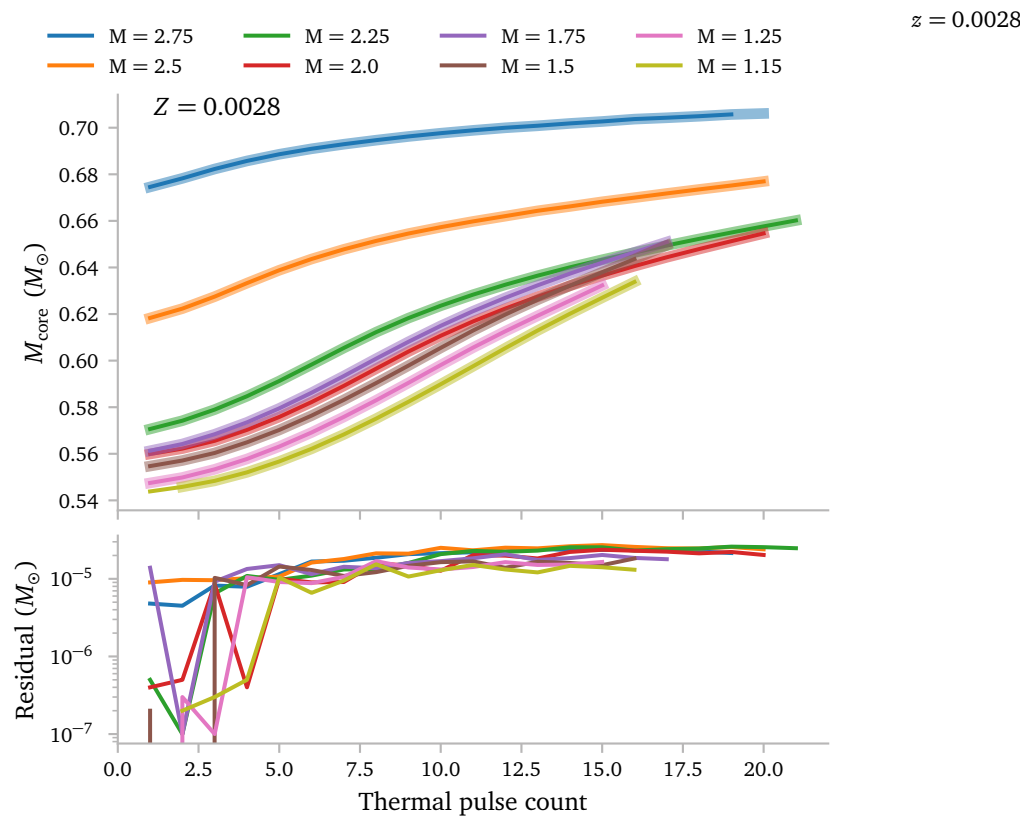


We see, quite unsurprisingly, that the amount of accreted barium increases with mass.

References

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. *ApJ***825**, 26.

A Additional plots



B Abundance Code

```
class Isotope:
    def __init__(self, isotope):
        self.key = isotope

        match isotope:
            case "n":
                self.mass = 1
                self.name = f"${\\textrm{{{isotope}}}}$"
                self.short_name = "neutron"
            case "p":
                self.mass = 1
                self.name = f"${\\textrm{{{isotope}}}^{{{+}}}}$"
                self.short_name = "proton"
            case "d":
                self.mass = 2
                self.name = f"${\\textrm{{{isotope}}}}$"
                self.short_name = "deuterium"
            case _:
                match = re.match(r"([a-zA-Z]+)(\\d+)$", isotope)
                if match:
                    self.short_name = match.group(1)
                    self.mass = int(match.group(2))
                    self.name = (
                        f"${\\textrm{{{self.short_name.capitalize()}}}_{{{self.mass}}}}$"
                    )
                else:
                    self.short_name = None
                    self.name = None
                    self.mass = None

    def __str__(self):
        return f"{self.key} with mass {self.mass}"

    def __repr__(self):
        return f"{self.key}"

class Element:
    def __init__(self, name):
        self.key = name
        self.name = name.capitalize()
        self.isotopes = {}

    def add_isotope(self, isotope):
        self.isotopes[isotope.key] = isotope

    def __getitem__(self, isotope):
        return self.isotopes[isotope]

    def __iter__(self):
        return iter(self.isotopes.values())

    def __repr__(self):
        return self.key

    def __getattr__(self, name):
        if name in self.isotopes:
            return self.isotopes[name]

class AbundanceTables:
    def __init__(self):

        with open("data/intershell_pd_df.pkl", "rb") as f:
            self.intershell = pickle.load(f)

        with open("data/env_pd_df.pkl", "rb") as f:
            self.envelope = pickle.load(f)

        with open("data/tp_info_pd_df.pkl", "rb") as f:
            self.tp = pickle.load(f)

        species = self.intershell.columns[4:]

        self.isotopes = {spec: Isotope(spec) for spec in species}
        self.elements = {}

        for isotope in self.isotopes.values():
            if isotope.short_name is None:
                continue

            element = self.elements.setdefault(
                isotope.short_name, Element(isotope.short_name)
            )
            element.add_isotope(isotope)

        self.envelope = self.envelope[self.envelope["pmz"] == 2e-3]
        self.envelope = self.envelope[self.envelope["N_ov"] != 0]
        self.envelope = self.envelope[self.envelope["Z"].astype(np.float64) == 0.014]
        self.envelope = self.envelope[self.envelope["M_init"].astype(np.float64) == 2]

    def __getattr__(self, name):
        try:
            return self.isotopes[name]
        except KeyError:
            raise AttributeError(f"{type(self).__name__} has no attribute {name!r}")

    def get_initial_envelope_abundance(self, element, metallicity):
        df = self.envelope[self.envelope["element"] == element]
        return float(df["massfrac"].iloc[-1]) * metallicity

    def get_pulse_info_specific_model(self, mass, metallicity):
        df = self.tp[self.tp["initial_mass"] == mass]
        df = df[df["z"] == metallicity]
        return df
```



```
class Abundances:
    """
    this class computes and contains the abundances of the envelope of a
    binary run. it uses the MESA binary simulation, combined with the
    simple binary to determine
    """

    def __init__(self, model, df):
        self.model = model
        df_mix = df.intershell[df.intershell["pmz"] == "2e-3"]
        self.df = df

        simple = get_star(full_path=m.params["single_star"])

        self.dup_simple = compute_m_DUP(simple)
        self.dup_detailed = compute_m_DUP(model, self.dup_simple)

        binary_start_age = model.age[0]

        simple_age = np.asarray(simple.age)

        self.simple_end_idx = (
            np.searchsorted(
                simple_age,
                binary_start_age,
                side="right",
            )
            - 1
        )

        self.total_length = self.simple_end_idx + len(self.model.age)
        self.m_dup = np.zeros(self.total_length)
        self.m_env = np.concatenate(
            [simple.m_env[: self.simple_end_idx], self.model.envelope_mass]
        )
        self.time = np.concatenate([simple.age[: self.simple_end_idx], self.model.age])

        m2 = np.concatenate(
            [
                self.model.sb.m2[: self.simple_end_idx],
                self.model.star_2_mass,
            ]
        )

        valid = ~np.isnan(m2)

        m2_filled = np.nan_to_num(m2, nan=0)

        dm = np.diff(m2_filled)
        dm[~valid[:-1] | ~valid[1:]] = 0

        self.dm_acc = np.concatenate([[0], np.clip(dm, 0, np.inf)])

        for key, value in self.dup_simple.items():
            if value["index"] > self.simple_end_idx:
                break
            self.m_dup[value["index"]] = value["mass"]

        for key, value in self.dup_detailed.items():
            self.m_dup[self.simple_end_idx + value["index"]] = value["mass"]

    def __getattr__(self, name):
        if name in self.df.elements:

            # compute the intershell elemental abundance
            intershell = self.compute_intershell(name)

            # compute the initial envelope abundance
            envelope = self.compute_envelope_abundance(name, intershell)

            self.df.elements[name].envelope = envelope
            self.df.elements[name].intershell = intershell
            self.df.elements[name].m_accreted = np.cumsum(envelope * self.dm_acc)

            return self.df.elements[name]

        if name in self.df.isotopes:

            return self.df.isotopes[name]

    def compute_intershell(self, name):

        intershell = np.zeros(self.total_length)
        for isotope in self.df.elements[name].isotopes:

            print(isotope)
            intershell += self.get_abundance(
                self.df.isotopes[isotope],
                self.model.params["m"],
                self.model.params["z"],
                np.cumsum(self.m_dup + 1e-12),
            )
        return intershell

    def get_iso_and_Mdredge(self, isotope, intershell, tp_info):
        # keep only pulses that exist in both datasets
        intershell = intershell[intershell.ntp.isin(tp_info.pulse)]

        # one intershell abundance per pulse
        i_data = intershell.drop_duplicates("ntp").set_index("ntp")

        # one dredge-up mass per pulse
        tp_data = tp_info.drop_duplicates("pulse").set_index("pulse")

        # common pulses, in ascending order
        pulses = i_data.index.intersection(tp_data.index).sort_values()

        Mdredge = tp_data.loc[pulses, "Ddredge"].to_numpy()
        isotope_abundance = i_data.loc[pulses, isotope.key].to_numpy()

        Mdredge = np.log10(np.cumsum(Mdredge) + 1e-12)
        isotope_abundance = np.log10(isotope.mass * isotope_abundance + 1e-20)

        return isotope_abundance, Mdredge

    def get_abundance(self, isotope, M, Z, Mdredge_interp, drop=None):

        if Z in [0.0028, 0.007, 0.014]:
            return 10 ** self.get_abundance_Z(isotope, M, Z, Mdredge_interp, drop)

        if Z <= 0.0028:
            return 10 ** self.get_abundance_Z(isotope, M, 0.0028, Mdredge_interp, drop)
        if Z >= 0.014:
            return 10 ** self.get_abundance_Z(isotope, M, 0.014, Mdredge_interp, drop)

        if Z <= 0.007:
            z_min = 0.0028
            z_max = 0.007
        else:
            z_min = 0.007
            z_max = 0.014

        abundance_min = self.get_abundance_Z(isotope, M, z_min, Mdredge_interp, drop)
        abundance_max = self.get_abundance_Z(isotope, M, z_max, Mdredge_interp, drop)
        weight = (np.log10(Z) - np.log10(z_min)) / (np.log10(z_max) - np.log10(z_min))

        return 10 ** (abundance_min + weight * (abundance_max - abundance_min))

    def get_abundance_Z(self, isotope, M, Z, Mdredge_interp, drop=None):
        intershell_metal = self.df.intershell.query(
            f"Z == {Z} and pmz == 2e-3 and last == 1"
        ).sort_values("ntp")

        ms = np.unique(intershell_metal.M1tp)
        ms.sort()
        if drop != None:
            for i, m in enumerate(ms):
                if m == drop:
                    ms = np.delete(ms, i)

        try:
            arg_max = np.where(ms >= M)[0][0]
            i1_max = intershell_metal.query(f"M1tp == {ms[arg_max]}")
            tp1_max = self.df.tp.query(f"initial_mass == {ms[arg_max]} and z == {Z}")
            iso, Mdredge_real = self.get_iso_and_Mdredge(isotope, i1_max, tp1_max)
            abundance_max = np.interp(np.log10(Mdredge_interp), Mdredge_real, iso)
        except IndexError:
            abundance_max = None

        try:
            arg_min = np.where(ms < M)[0][-1]
            i1_min = intershell_metal.query(f"M1tp == {ms[arg_min]}")
            tp1_min = self.df.tp.query(f"initial_mass == {ms[arg_min]} and z == {Z}")
            iso, Mdredge_real = self.get_iso_and_Mdredge(isotope, i1_min, tp1_min)
            abundance_min = np.interp(np.log10(Mdredge_interp), Mdredge_real, iso)
        except IndexError:
            abundance_min = None

        if type(abundance_min) == type(None):
            return abundance_max
        if type(abundance_max) == type(None):
            return abundance_min
        weight = (M - ms[arg_min]) / (ms[arg_max] - ms[arg_min])

        return abundance_min + weight * (abundance_max - abundance_min)

    def compute_envelope_abundance(self, name, intershell):
        # INFO: gets the initial envelope abundance of the element
        # scaled by the metallicity of the model.
        initial_envelope_abundance = self.df.get_initial_envelope_abundance(
            element=name,
            metallicity=self.model.params["z"],
        )

        # INFO: computes the elemental abundance in the envelope by
        # enriching it with intershell abundances.
        envelope = np.zeros(self.total_length)
        delta_M_element = intershell * self.m_dup
        for i in range(self.total_length):
            if i == 0:
                envelope[i] = initial_envelope_abundance
                continue
            envelope[i] = (envelope[i - 1] * self.m_env[i] + delta_M_element[i]) / (
                self.m_env[i] + self.m_dup[i]
            )

        return envelope
```